ESQUILO INVEST
Pocket Ops

POCKET OPS V2.0.0

Arquitetura do MVP Mobile e integração Flutter + AppScript

Documento visual de acompanhamento da release **Pocket Ops v2.0.0**, consolidando a estrutura técnica do app Flutter, a ponte com o AppScript e o recorte funcional entregue para teste local no Android.

Flutter mobile real | Backend em AppScript | APK debug Android | MVP sem nova regra de negócio

RELEASE PRINCIPAL
Pocket Ops v2.0.0

ESCOPO
Migração do dashboard web para app mobile Flutter

BACKEND
AppScript publicado como Web App em `/exec`

PLATAFORMA-ALVO
Android local com APK debug para validação rápida

VISÃO GERAL

Solução pensada para migrar com segurança

O app Flutter replica a leitura principal do produto atual em um fluxo mobile real, sem duplicar regra de negócio e sem substituir o backend existente. O AppScript continua centralizando os cálculos, leituras e payloads; o Flutter organiza navegação, renderização de cards táticos e consumo HTTP para Android.

4
Blocos-chave no app: navegação, estado, modelos e serviços.

1
Endpoint base reutilizado no AppScript, com recursos JSON por `resource`.

0

Novas regras de negócio adicionadas nesta release.

ESCOPO DO MVP

O que entra agora

HOME Patrimônio total, performance, score da carteira, alocação, radar e missão do mês em leitura vertical.

DETALHE Navegação simples para categorias, preservando cada bloco como unidade independente.

IA Estrutura pronta para a Esquilo IA, consumindo o mesmo backend quando o endpoint estiver acessível ao app.

DIAGRAMA

Fluxo técnico da arquitetura

O mobile consome o AppScript por HTTP, interpreta envelopes JSON e transforma o payload em modelos Dart usados por telas e widgets independentes.

Flutter App

`PocketOpsApp`, `DashboardScreen`, tabs e detalhe por categoria.

Service Layer

`AppScriptDashboardService` monta a URL, chama o backend e normaliza erros.

AppScript Web App

`/exec?format=json&resource=...` responde dashboard, IA e health.

Dados e cálculo

Planilhas, métricas e motor analítico continuam centralizados no backend atual.

CAMADAS

Separação por responsabilidade

APP

Shell do aplicativo, tema, roteamento e bootstrap.

FEATURES

Home, detalhe de categoria, insights e controladores.

SERVICES

Cliente HTTP para AppScript e tratamento de resposta remota.

MODELS

Estruturas Dart que espelham o payload do backend sem criar regra paralela.

DECISÕES

Principais escolhas desta release

- Aplicar a URL real do AppScript no app para simplificar o teste local.
- Manter `--dart-define` como override sem desmontar a configuração existente.
- Preservar o AppScript como backend principal, sem duplicação de lógica.
- Isolar a home em cards táticos legíveis, otimizados para leitura vertical.
- Tratar HTML de login do Google como erro explícito de integração, e não como falha genérica de parse.

INTEGRAÇÃO

Conexão Flutter + AppScript

A base URL ativa desta execução foi aplicada diretamente na configuração do app. O objetivo é garantir um APK debug pronto para validar a integração real no Android sem depender de placeholders.

ENDPOINT BASE APLICADO

https://script.google.com/macros/s/AKfycbya0mKpePyZV79S3ta0hJf0fhhwWfeUFwfopMOsZRSuqT7EF8LiTz6dV2suOuJNWve-/exec

PONTO	IMPLEMENTAÇÃO ATUAL	OBSERVAÇÃO OPERACIONAL
Base URL	`AppEnvironment.appScriptBaseUrl`	Usa a URL real como padrão e ainda aceita override por `--dart-define`.
Chamadas remotas	`AppScriptDashboardService`	Monta query com `format=json` e `resource=dashboard|health|ai-analysis`.
Tratamento de erro	Exceções de integração normalizadas no app	Quando o backend devolve HTML de login, a mensagem aponta o problema de publicação/acesso.

STATUS DESTA EXECUÇÃO

A URL foi aplicada corretamente no Flutter e o build Android segue íntegro. Na validação remota, porém, o endpoint publicado em `/exec` respondeu uma página HTML de login do Google em vez de JSON, indicando bloqueio de acesso do Web App para o app mobile nativo.

MAPA DE ARQUIVOS

Blocos principais do projeto mobile

ARQUIVO	PAPEL NO MVP
`lib/core/config/app_environment.dart`	Release, versão e URL base do AppScript.
`lib/services/app_script_dashboard_service.dart`	Chamada HTTP, envelope JSON e tratamento de falhas remotas.
`lib/features/dashboard/dashboard_screen.dart`	Home principal com visão operacional, carteira e insights.
`lib/models/dashboard_payload.dart`	Modelos Dart do dashboard e dados das categorias.
`android/app/src/main/AndroidManifest.xml`	Permissão de internet para o APK consumindo o backend.

CONCLUSÃO EXECUTIVA

MVP pronto para validação prática

A release **Pocket Ops v2.0.0** já organiza a base do app Flutter, conecta o consumo ao AppScript e prepara o Android para teste local com APK debug. O ponto crítico de operação deixa de ser o app e passa a ser a acessibilidade do Web App publicado.

DOCUMENTO VISUAL DE ACOMPANHAMENTO DO MVP MOBILE